

[CPE223] COMPUTER ARCHITECTURES

M3 Mock Final Exam

Topic: Memory System, Cache Memory, Virtual Memory and Disk

Time: 2 Hours | Full Score: 50 Points

Part 1: Multiple Choice Questions (20 Points)

Instructions: Select the correct answer. (1 point each)

1. In the memory hierarchy, what generally happens when moving from CPU registers/cache toward disk storage?
 - a) Speed increases, size decreases, and cost per bit increases.
 - b) Speed decreases, size increases, and cost per bit decreases.
 - c) Speed, size, and cost per bit all increase.
 - d) Access time is the same for all memory levels.
2. Which statement best describes **temporal locality**?
 - a) Recently accessed data or instructions are likely to be accessed again soon.
 - b) Addresses close to a recently accessed address are likely to be accessed soon.
 - c) A page table entry must always be stored in the TLB.
 - d) The disk head always moves to the closest track first.
3. Which statement best describes **spatial locality**?
 - a) Recently executed instructions are never executed again.
 - b) Addresses close to a recently accessed address are likely to be accessed soon.
 - c) The least recently used cache block should never be replaced.
 - d) Cache miss penalty is always zero for sequential programs.
4. A direct-mapped cache has a 32-bit byte address, total cache size 16 KB, and block size 32 bytes. What are the tag, index, and block-offset sizes?
 - a) Tag = 16 bits, Index = 11 bits, Offset = 5 bits
 - b) Tag = 17 bits, Index = 10 bits, Offset = 5 bits
 - c) Tag = 18 bits, Index = 9 bits, Offset = 5 bits
 - d) Tag = 19 bits, Index = 8 bits, Offset = 5 bits

5. A 64 KB cache is 4-way set associative with 64-byte blocks and a 32-bit byte address. How many set-index bits are required?
- a) 6 bits
 - b) 7 bits
 - c) 8 bits
 - d) 9 bits
6. Using $t_{ave} = ht_C + (1 - h)t_M$, a cache has hit rate $h = 0.95$, cache access time $t_C = 1$ cycle, and miss time $t_M = 21$ cycles. What is the average access time?
- a) 1.05 cycles
 - b) 1.50 cycles
 - c) 2.00 cycles
 - d) 2.95 cycles
7. Which statement correctly describes a **write-back** cache protocol?
- a) Cache and main memory are updated simultaneously on every write.
 - b) The processor bypasses cache and writes only to disk.
 - c) The cache block is updated and marked dirty; main memory is updated later when the block is replaced.
 - d) The TLB is flushed every time the cache writes data.
8. In the LRU replacement algorithm, which cache block is selected for replacement?
- a) The block with the smallest tag value.
 - b) The block most recently referenced.
 - c) The block that has gone the longest time without being referenced.
 - d) A random block from main memory.
9. What usually happens on a **read miss** in a cache?
- a) The missing block is loaded from lower-level memory into cache, then the requested word is supplied to the CPU.
 - b) The CPU writes the requested word to disk immediately.
 - c) The cache ignores the miss and returns zero.
 - d) The page table is deleted to reduce memory usage.

10. What is the primary purpose of a Translation Lookaside Buffer (TLB)?

- a) To cache recently used virtual-to-physical address translations.
- b) To store all program data permanently after power off.
- c) To replace the cache data array in a write-through cache.
- d) To compute disk rotational latency.

11. Use this same system for Questions 11–20. A byte-addressable machine uses 32-bit virtual addresses, 28-bit physical addresses, and 4 KB pages. A 1024×1024 array A of 32-bit integers starts at virtual address $0x00000000$ and is stored in **column-major order**.

$$VA(A[i][j]) = (j \times 1024 + i) \times 4$$

The MMU/TLB state is:

TLB		Page Table			Disk Info		
VPN	PPN	VPN	Location	Valid	Disk page	Start block	Block size
		0x00001	PPN 0x003A	1			
0x00003	0x0006	0x00002	Disk D5	0			
0x00007	0x0012	0x00003	PPN 0x0006	1			
		0x00004	PPN 0x00B0	1			
					D5	0x2400	512 bytes

How many bytes are used by the whole array?

- a) 1 MB
- b) 2 MB
- c) 4 MB
- d) 8 MB

12. What is the virtual address of $A[0][1]$?

- a) $0x00000004$
- b) $0x00000400$
- c) $0x00001000$
- d) $0x00002000$

13. How many virtual pages are occupied by the whole array?

- a) 256 pages
- b) 512 pages
- c) 1024 pages
- d) 4096 pages

14. For virtual address $0x00002C10$, what are the virtual page number (VPN) and page offset?
- a) VPN = $0x00001$, Offset = $0xC10$
 - b) VPN = $0x00002$, Offset = $0xC10$
 - c) VPN = $0x00002$, Offset = $0x2C1$
 - d) VPN = $0x0002C$, Offset = $0x010$
15. According to the TLB and page table, what happens when the CPU references virtual address $0x00002C10$?
- a) TLB hit and the physical address is $0x00002C10$.
 - b) TLB miss, page table valid, no page fault.
 - c) TLB miss followed by page fault.
 - d) The cache line is marked dirty immediately.
16. Which disk block contains byte offset $0xC10$ within disk page D5?
- a) $0x2404$
 - b) $0x2405$
 - c) $0x2406$
 - d) $0x2408$
17. How many disk blocks must be read to load the whole 4 KB page?
- a) 4 blocks
 - b) 8 blocks
 - c) 12 blocks
 - d) 16 blocks
18. After the missing page is loaded into physical frame PPN $0x0020$, what is the physical address for virtual address $0x00002C10$?
- a) $0x00002C10$
 - b) $0x00020C10$
 - c) $0x00200C10$
 - d) $0x00202C10$
19. For virtual address $0x00003204$, what is the TLB result and physical address?
- a) TLB miss; $0x00003204$
 - b) TLB hit; $0x00006204$
 - c) TLB hit; $0x00006C10$
 - d) Page fault; no physical address yet

20. In the normalization code from the written practice, each column is one 4 KB page and loading one page costs 40 ms. If the array is stored in column order, what is the estimated total page-fault time?

- a) 10.24 s
- b) 20.48 s
- c) 40.96 s
- d) 81.92 s

Part 2: Short-answer questions (10 Points)

Instructions: Briefly answer or explain the following concepts. (1 point each)

1. What is the **benefit of cache memory**?
2. Is it possible for a program to have a **100% cache miss rate**? Briefly explain.
3. What is the purpose of a **tag store** in cache memory?
4. Briefly distinguish **temporal locality** and **spatial locality**.
5. In cache addressing, what are **tag**, **index/set**, and **offset** used for?
6. Write the full names of these abbreviations: **TLB**, **LRU**, and **AMAT**.
7. State one key difference between **write-through** and **write-back** cache protocols.
8. What is **demand paging**, and when does a page fault occur?
9. What is the difference between a **virtual address** and a **physical address**?
10. What are the main components of **disk access latency**?

Part 3: Problem Solving (20 Points)

Instructions: Show your work in detail.

1. (10 Points) Array Normalization, Direct-Mapped Cache, and Page Faults

A 1024×1024 array of 32-bit numbers is stored in the main memory starting at address 0. The array is to be normalized as follows. For each column, the largest value is found and all elements of the column are divided by this maximum value. Assume that each page in the direct-mapped cache consists of 4 KB, and the cache size is 1 MB.

- 1.1 (2 points) How many bytes are used by the whole array?
- 1.2 (2 points) How many array elements fit in one 4 KB page?
- 1.3 (2 points) How many 4 KB pages can fit in the 1 MB cache?
- 1.4 (2 points) How many page faults would occur if the elements of the array are stored in **column order** in the main memory?
- 1.5 (2 points) Suppose it takes 40 ms to load a page from the main memory to cache and the time to load data from cache is very low. Estimate the total time needed to perform this normalization.

2. (10 Points) Set-Associative Cache with LRU

This computer system uses set-associative (2 sets) mapping for cache memory with LRU. Two figures show the source code and the original cache content after running the code (captured after pass: $j = 3, 7, 9$ and $i = 4, 2, 0$).

Assume the data cache has 8 block positions total. Set 0 contains block positions 0–3, and Set 1 contains block positions 4–7. Use the same cache-mapping idea as the practice exam table. Array $A(4, 10)$ is stored in column-major order, and the accessed elements $A(0, j)$ for $j = 0, 1, \dots, 9$ are placed in Set 0. Set 1 remains empty.

```
SUM := 0
for j := 0 to 9 do
    SUM := SUM + A(0, j);
end
AVE := SUM / 10;

for i := 9 downto 0 do
    A(0, i) := A(0, i) / AVE;
end
```

Changed line: for i := 9 downto 0 do

The following table shows the original cache content after running the code above, captured after pass $j = 3, 7, 9$ and $i = 4, 2, 0$.

Set	Block pos.	$j = 3$	$j = 7$	$j = 9$	$i = 4$	$i = 2$	$i = 0$
Set 0	0	$A(0, 0)$	$A(0, 4)$	$A(0, 8)$	$A(0, 4)$	$A(0, 4)$	$A(0, 0)$
	1	$A(0, 1)$	$A(0, 5)$	$A(0, 9)$	$A(0, 5)$	$A(0, 5)$	$A(0, 1)$
	2	$A(0, 2)$	$A(0, 6)$	$A(0, 6)$	$A(0, 6)$	$A(0, 2)$	$A(0, 2)$
	3	$A(0, 3)$	$A(0, 7)$	$A(0, 7)$	$A(0, 7)$	$A(0, 3)$	$A(0, 3)$
Set 1	4						
	5						
	6						
	7						

Now change exactly the highlighted line of code to:

for i := 9 downto 0 do $\Rightarrow$ for i := 0 to 9 do

What should the content of cache be? Draw/fill the new table.

Set	Block pos.	$j = 3$	$j = 7$	$j = 9$	$i = 4$	$i = 2$	$i = 0$
Set 0	0						
	1						
	2						
	3						
Set 1	4						
	5						
	6						
	7						

Answer Key (เฉลยละเอียด)

Part 1: Multiple Choice Answers

ข้อที่	เฉลย	คำอธิบายภาษาไทย
1	B	ยิ่งใกล้ CPU มากขึ้น หน่วยความจำมักใหญ่ขึ้นและถูกลบต่อบิต แต่ช้าลง เช่น จาก cache ไป RAM และ disk
2	A	Temporal locality คือสิ่งที่เพิ่งถูกใช้งาน มีโอกาสถูกใช้อีกในเวลาใกล้ ๆ
3	B	Spatial locality คือถ้าเข้าถึง address หนึ่งแล้ว address ที่อยู่ใกล้กันมักถูกใช้ต่อ
4	C	Offset = $\log_2 32 = 5$ bits, cache lines = $16KB/32 = 512 = 2^9$, tag = $32 - 9 - 5 = 18$ bits
5	C	จำนวน block = $64KB/64 = 1024$, จำนวน set = $1024/4 = 256 = 2^8$ จึงใช้ index 8 bits
6	C	$t_{ave} = 0.95(1) + 0.05(21) = 2.00$ cycles
7	C	Write-back เขียนลง cache ก่อนและตั้ง dirty bit จากนั้นค่อยเขียนกลับ main memory ตอน block ถูกแทนที่
8	C	LRU เลือก block ที่ไม่ได้ถูกอ้างอิงนานที่สุดออกก่อน
9	A	Read miss ต้องดึง block จาก memory ขึ้นล่างขึ้น cache แล้วจึงส่ง word ที่ต้องการให้ CPU
10	A	TLB เป็น cache เฉพาะทางสำหรับเก็บการแปลง virtual page ไป physical frame
11	C	Array มี 1024×1024 elements และแต่ละ element = 4 bytes ดังนั้นใช้ $1024 \times 1024 \times 4 = 4$ MB
12	C	Column-major: A[0][1] อยู่หลัง column แรก 1024 integers จึงเป็น $1024 \times 4 = 4096 = 0x00001000$
13	C	Page size = 4KB และ array size = 4MB ดังนั้นจำนวน page คือ $4MB/4KB = 1024$ pages
14	B	Page size 4KB ทำให้ offset มี 12 bits ดังนั้น $0x00002C10$ มี VPN = $0x00002$, offset = $0xC10$
15	C	VPN $0x00002$ ไม่อยู่ใน TLB และ page table มี valid bit = 0 อยู่บน Disk D5 จึงเป็น TLB miss แล้วเกิด page fault
16	C	Disk block size = $0x200$ bytes; $\lfloor 0xC10/0x200 \rfloor = 6$ ดังนั้น block คือ $0x2400+6 = 0x2406$
17	B	4KB page ต้องอ่าน $4096/512 = 8$ disk blocks เพื่อโหลดทั้งหน้า
18	B	PPN $0x0020$ รวมกับ offset $0xC10$ ได้ physical address $0x00020C10$
19	B	$0x00003204$ มี VPN $0x00003$ และ offset $0x204$; TLB มี VPN นี้อยู่แล้ว จึงได้ PPN $0x0006$ และ physical address $0x00006204$
20	C	Column-major ทำให้หนึ่ง column = 1024 integers = 4KB = 1 page; มี 1024 columns จึงใช้เวลา $1024 \times 40ms = 40.96s$

Part 2: Short Answer Solutions

- Benefit of cache:** cache ช่วยลด average memory access time โดยเก็บข้อมูลหรือคำสั่งที่ถูกใช้บ่อย/มีโอกาสถูกใช้ใกล้ ๆ CPU ทำให้ access ส่วนใหญ่เร็วกว่าไปอ่านจาก main memory
- 100% cache miss rate:**
เป็นไปได้ เช่น access ข้อมูลไม่ซ้ำหรือไม่มี locality
หรือ working set ใหญ่กว่า cache จน block ถูกแทนที่ก่อนใช้ซ้ำ
แต่โปรแกรมจริงมักมี locality จึงไม่ค่อยเกิดตลอดเวลา
- Tag store:** ใช้เก็บ tag และสถานะของ cache block เช่น valid/dirty เพื่อบอกว่า block ใน cache เป็นสำเนาของ memory block ใด และใช้ตรวจว่า access นั้นเป็น hit หรือ miss
- Temporal vs spatial locality:** temporal คือข้อมูล/คำสั่งที่เพิ่งใช้มีโอกาสดูถูกใช้อีกเร็ว ๆ นี้ ส่วน spatial คือถ้าใช้ address หนึ่งแล้ว

address ใกล้เคียงมักถูกใช้ตามมา

5. **Tag/index/offset**: tag ใช้ตรวจว่า block ไหนอยู่ใน cache, index/set ใช้เลือกตำแหน่งหรือ set ใน cache, offset ใช้เลือก byte/word ภายใน block
6. **Abbreviations**: TLB = Translation Lookaside Buffer, LRU = Least Recently Used, AMAT = Average Memory Access Time
7. **Write-through vs write-back**: write-through เขียน cache และ main memory พร้อมกัน ส่วน write-back เขียน cache ก่อนและค่อยเขียนกลับภายหลังเมื่อ block ถูกแทนที่
8. **Demand paging**: โหลด page เมื่อมีการอ้างถึงจริงเท่านั้น; page fault เกิดเมื่อ page ที่ต้องใช้ยังไม่อยู่ใน physical memory
9. **Virtual vs physical address**: virtual address เป็น address ที่โปรแกรมเห็น ส่วน physical address เป็นตำแหน่งจริงใน RAM หลังผ่าน MMU/page table
10. **Disk access latency**: โดยทั่วไปประกอบด้วย seek time, rotational latency และ transfer time

Part 3: Problem Solving Solutions

1. Array Normalization, Direct-Mapped Cache, and Page Faults

- 1.1 Array มี 1024×1024 elements และแต่ละ element ขนาด 32 bits = 4 bytes

$$1024 \times 1024 \times 4 = 4,194,304 \text{ bytes} = 4 \text{ MB}$$

- 1.2 Page size = 4KB = 4096 bytes ดังนั้นจำนวน elements ต่อ page คือ

$$4096/4 = 1024 \text{ elements/page}$$

- 1.3 Cache size = 1MB = 2^{20} bytes และ page/block size = 4KB = 2^{12} bytes

$$2^{20}/2^{12} = 2^8 = 256 \text{ pages}$$

- 1.4 ถ้าเก็บแบบ column order หนึ่ง column มี 1024 elements

$$1024 \times 4 = 4096 \text{ bytes} = 1 \text{ page}$$

การ normalize ทำทีละ column ดังนั้นแต่ละ column ทำให้เกิด page fault ประมาณ 1 ครั้ง และมีทั้งหมด 1024 columns

$$\text{page faults} = 1024$$

- 1.5 ถ้าโหลดหนึ่ง page ใช้ 40 ms

$$1024 \times 40 \text{ ms} = 40960 \text{ ms} = 40.96 \text{ s}$$

2. Set-Associative Cache with LRU

Changed line in the code

```
for i := 9 downto 0 do  $\Rightarrow$  for i := 0 to 9 do
```

Important note

หัวตารางยังคงอิงตามรูปประกอบคือ $j = 3, 7, 9$ และ $i = 4, 2, 0$ แต่หลังเปลี่ยนโค้ดแล้ว loop ที่สองทำงานจริงตามลำดับ $i = 0, 1, \dots, 9$. ดังนั้นเวลาคิด LRU ต้องไล่ตามลำดับการรันจริงก่อน แล้วจึงนำสถานะ cache หลัง pass $i = 4, i = 2$, และ $i = 0$ ไปใส่ในคอลัมน์ที่กำหนด.

Completed cache-content table

Set	Block pos.	$j = 3$	$j = 7$	$j = 9$	$i = 4$	$i = 2$	$i = 0$
Set 0	0	$A(0, 0)$	$A(0, 4)$	$A(0, 8)$	$A(0, 2)$	$A(0, 2)$	$A(0, 8)$
	1	$A(0, 1)$	$A(0, 5)$	$A(0, 9)$	$A(0, 3)$	$A(0, 9)$	$A(0, 9)$
	2	$A(0, 2)$	$A(0, 6)$	$A(0, 6)$	$A(0, 4)$	$A(0, 0)$	$A(0, 0)$
	3	$A(0, 3)$	$A(0, 7)$	$A(0, 7)$	$A(0, 1)$	$A(0, 1)$	$A(0, 7)$
Set 1	4	-	-	-	-	-	-
	5	-	-	-	-	-	-
	6	-	-	-	-	-	-
	7	-	-	-	-	-	-

Checking note: hit/miss result for the modified loop

หลังจบ loop แรกที่ $j = 9$ ใน Set 0 จะเหลือ $A(0, 8), A(0, 9), A(0, 6), A(0, 7)$ อยู่ใน cache แต่ loop ใหม่เริ่มจาก $i = 0$ ขึ้นไป จึงเกิด miss ต่อเนื่อง:

i	0	1	2	3	4	5	6	7	8	9
Result	M	M	M	M	M	M	M	M	M	M

LRU explanation

ตอนจบ loop แรก ข้อมูลที่ใหม่ที่สุดคือกลุ่มท้าย ๆ เช่น $A(0, 8)$ และ $A(0, 9)$. ถ้าใช้ loop เดิมแบบ `downto` จะเริ่มแตะข้อมูลท้าย ๆ ก่อน จึงมีโอกาส hit. แต่เมื่อเปลี่ยนเป็น $i = 0$ to 9 จะเริ่มจากข้อมูลต้น ๆ ที่ไม่อยู่ใน cache แล้วค่อย ๆ แทนที่ block เก่าตาม LRU ก่อนจะวนมาถึงข้อมูลท้าย ๆ ทำให้ใน loop ที่สองทุก access เป็น miss.